

Spring Framework – Day 20 Notes

Annotations – Day 03: Optional vs Mandatory Dependency Injection, Autowiring & @Autowired

1. Optional and Mandatory Dependency Injection

1.1 Setter Injection is Optional

- Setter injection means we add the dependency to the target class using a setter method.
- If the class provides a setter, but we do not add a <property> tag for it in the configuration file, we will not get any exception — the object is still created.

```
package com.day20;

import lombok.AllArgsConstructor;
import lombok.NoArgsConstructor;
import lombok.Setter;
import lombok.ToString;

@AllArgsConstructor
@NoArgsConstructor
@Setter
@ToString
public class A {
    int a;
}
```

application-context.xml

```
<bean id="a" class="com.day20.A">
    <!-- Even if we don't write any <property> tag,
         the object is still created -->
</bean>
```

1.2 Constructor Injection is Mandatory

- Constructor injection means we add the dependency to the target class using a constructor.
- If the class has a parameterized constructor, we **must** write the corresponding constructor injection in the configuration file.

```
package com.day20;

import lombok.AllArgsConstructor;
import lombok.NoArgsConstructor;
import lombok.Setter;
import lombok.ToString;

@AllArgsConstructor
@NoArgsConstructor
@Setter
@ToString
public class A {
    int a;
}
```

application-context.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
    xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        https://www.springframework.org/schema/context/spring-context.xsd">

    <bean id="a" class="com.day20.A">
        <!-- <constructor-arg value="10"></constructor-arg -->
        <!-- Without this constructor-arg we will get an exception -->
    </bean>
</beans>
```

Key takeaway

Setter-based dependencies are optional to declare in the XML file. Constructor-based dependencies are mandatory to declare, because the constructor cannot be called without its required arguments.

2. Autowiring in XML Configuration

Consider the following classes:

```
class Student {
    Course course;

    public void setCourse(Course course) {
        this.course = course;
    }

    public String toString() {
        return "[course = " + course + "]";
    }
}

class Course {
    private String courseName;

    public void setCourseName(String courseName) {
        this.courseName = courseName;
    }
}
```

application-context.xml

```
<beans>
    <bean id="course" class="com.day20.Course">
        <property name="courseName" value="Spring framework"></property>
    </bean>

    <!-- The IOC container creates the objects of every bean declared
        here. After creating an object, it injects all the
```

```

        dependencies present inside the class. -->

<bean id="student" class="com.day20.Student">
    <property name="course" ref="course"></property>
</bean>
</beans>

ApplicationContext context =
    new ClassPathXmlApplicationContext("application-context.xml");
// The IOC container reads the XML file

```

2.1 What if we don't want to write the ref ourselves?

- Suppose we do not want to write the dependency reference (**ref**) by ourselves, and instead want the IOC container to do it automatically. For this, we change the bean definition to use the **autowire** attribute.

```

<bean id="student" class="com.day20.Student" autowire="mode">
    <!-- <property name="course" ref="course"></property> -->
</bean>

<!-- "mode" represents:
    - How to identify the dependency
    - How to inject the dependency -->

```

2.2 What is Autowire?

- Instead of declaring the dependency between bean definitions ourselves, the IOC container itself takes care of identifying and injecting the dependency.
- Autowire mode is of three types:

```

byName
- Identifies the dependency : by matching the attribute name
- Injects the dependency    : with the help of setter injection

byType
- Identifies the dependency : by matching the attribute's class type
- Injects the dependency    : with the help of setter injection

constructor
- Identifies the dependency : by matching the constructor
- Injects the dependency    : with the help of constructor injection

```

3. The @Autowired Annotation

- The same thing we do inside the XML file with **autowire** can also be done without writing any XML, using **@Autowired**.
- By using **@Autowired**, we can manage the dependency without explicitly writing it in a place that may need to change frequently.
- One important difference: **@Autowired does not work with modes** (byName / byType / constructor) the way the XML autowire attribute does.
- How it identifies the dependency: always based on **byType**.

- How it injects the dependency: it injects at the exact place where `@Autowired` is written — that place identifies and receives the dependency.

3.1 Where Can `@Autowired` Be Written?

`@Autowired` can be written at 4 places:

- Fields
- Methods (setters)
- Constructors
- Arbitrary methods

3.2 Example: `@Autowired` on a Setter Method

```
public class Student {
    Course course;

    public Student() {
        System.out.println("Student class object");
    }

    public Course getCourse() {
        return course;
    }

    @Autowired(required = false)
    // It will work with byType, so when we write @Autowired on
    // the setter method, it checks the type of the setter's
    // parameter, searches the configuration file for a matching
    // bean definition, and if found, injects it through the
    // setter method (the same place where we wrote @Autowired).
    public void setCourse(Course course) {
        this.course = course;
    }

    @Override
    public String toString() {
        return "Student [course=" + course + "]";
    }
}
```

application-context.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        https://www.springframework.org/schema/context/spring-context.xsd">

    <bean id="course" class="com.day20.Course">
        <property name="name" value="spring framework"></property>
    </bean>
```

```

<bean id="student" class="com.day20.Student">
    <!-- <property name="course" ref="course"></property> -->
</bean>

<context:annotation-config></context:annotation-config>

</beans>

```

3.3 Can We Get the Student Object Without a Course Bean?

(Think about this without `@Autowired` first.)

- **Yes**, we can still get the object of the Student class, because setter injection is optional.

```

public class Student {
    Course course;

    public Student() {
        System.out.println("Student class object");
    }

    public Course getCourse() {
        return course;
    }

    public void setCourse(Course course) {
        this.course = course;
    }

    @Override
    public String toString() {
        return "Student [course=" + course + "]";
    }
}

```

application-context.xml (no Course bean present, only the Student bean):

```

<bean id="student" class="com.day20.Student">
</bean>

```

Output:

```

Student class object

```

3.4 Now Add `@Autowired` on the Setter. Still With No Course Bean

```

@Autowired
public void setCourse(Course course) {
    this.course = course;
}

```

application-context.xml (again, no Course bean present, only the Student bean):

```

<bean id="student" class="com.day20.Student">
</bean>

```

- Question: in the same way, can we get the object of Student here, since setter injection is normally optional?

- **No**, we will not get the object here, because `@Autowired` has a property called `@Autowired(required = true/false)`, and by default it is **true**. This makes autowiring mandatory at that point.

Note — @Required vs @Autowired

A related question: does `@Required` also make the setter mandatory, the same way `@Autowired` does?

`@Required` is written only at the setter (constructors have a circular-dependency issue with `@Required`), whereas `@Autowired` can be written at different places, and wherever it is written, that place becomes mandatory — meaning the bean definition for that dependency must exist.

3.5 Making @Autowired Optional

- If we want `@Autowired` to be optional, we set **required** to **false**:

```
@Autowired(required = false)
```

- If we try this out on the same program (still with no `Course` bean), we will be able to get the object of `Student`.

3.6 @Autowired at the Constructor Level

```
public class Student {
    Course course;

    public Student() {
        System.out.println("Student class object");
    }

    @Autowired
    public Student(Course course) {
        super();
        System.out.println("Through student constructor");
        this.course = course;
    }

    @Override
    public String toString() {
        return "Student [course=" + course + "]";
    }
}
```

application-context.xml

```
<bean id="course" class="com.day20.Course">
    <property name="name" value="spring framework"></property>
</bean>

<bean id="student" class="com.day20.Student">
    <!-- <property name="course" ref="course"></property> -->
</bean>
```

- Question: what happens if we remove the `Course` bean definition from the configuration?
- We get an **exception**, because `@Autowired` is **required = true** by default.
- So, making it optional makes the autowiring at the constructor level create the object even without the dependent class's bean definition.

```
@Autowired(required = false)
public Student(Course course) {
    super();
    System.out.println("Through student constructor");
    this.course = course;
}
```

application-context.xml (Course bean commented out / removed):

```
<!--
<bean id="course" class="com.day20.Course">
    <property name="name" value="spring framework"></property>
</bean>
-->

<bean id="student" class="com.day20.Student">
    <!-- <property name="course" ref="course"></property> -->
</bean>
```

- With this change, we successfully get the Student class object.

End of Day 20 notes — practice this content and read all points carefully before the next session.